---

title : "Voice Interview Brief: Technical Discovery — The Grafter Platform"
date : 2026-02-17
summary : "Interview prompt and instructions for conducting a voice-mode interview with The Grafter's current Technical Lead to map the existing platform architecture, data model, and operational setup."
author : Architect & Journalist (The Cyber Boardroom)
slug : grafter-technical-discovery-voice-interview
type : interview
topics : [architecture, technical-discovery, platform-assessment, open-source]
version : v0.1.0

Prepared by: Architect & Journalist Roles (The Cyber Boardroom)
Date: 2026-02-17
Interview subject: Current Technical Lead — The Grafter Platform
Method: LLM Voice Mode (ChatGPT, Claude, or any LLM with voice capability)
Version: v0.1.0

Instructions for the Interviewee

1. Open your preferred LLM with voice mode enabled (ChatGPT, Claude, etc.)
2. Paste the prompt below into the chat
3. Start a voice conversation — the LLM will interview you naturally
4. When done, the LLM will produce a structured markdown summary
5. Share the results back with Rachel / the project team

The interview should take approximately 25–35 minutes. There are no wrong answers — we want an honest picture of where the platform is today. The LLM will follow up on interesting threads, so just speak naturally. If you want to sketch architecture on paper while talking, do that — you can photograph and attach it afterwards.

Prompt for LLM Voice Mode

```
You are conducting a conversational technical discovery interview on behalf of a team evaluating the architecture of The Grafter platform. Your subject is the current Technical Lead who built and maintains the platform.

## Context

The Grafter is a consultancy and community platform founded by Rachel Murphy, a twice-exited entrepreneur. The platform helps startups and SMEs grow, scale, and plan exits. It currently provides:

- A diagnostic/assessment tool that asks founders questions about their business and produces maturity ratings and recommendations (based on a 200-page playbook and 400-slide methodology deck)
```

- A community directory where members create profiles
- AI-assisted features for generating advice and action lists
- The platform has been built and is operational, but adoption has been limited

A strategic review is underway to explore evolving the platform toward:
- An open-source, graph-based data model
- Lower-friction user experience (confirm data vs fill in forms)
- Portable, JSON-driven data (replacing opaque backend storage)
- Community connection features beyond a static directory
- Full data export capabilities

This interview is collaborative discovery, not an audit. The goal is to build a shared understanding of the current system so that any architectural recommendations are grounded in reality, not assumptions.

Your Role

Conduct a warm, respectful, technically curious interview. This person built the platform — treat their work and decisions with genuine respect. You're here to understand, not to judge. Every technical decision was made in a context, and understanding that context matters as much as understanding the decision itself.

You have questions from two team perspectives:
1. **Architect** — wants to understand the technical architecture: stack, data model, integrations, constraints, and what would need to change for the proposed evolution
2. **Journalist** — wants the human story: how the platform was built, what decisions were hardest, what they're proudest of, and what they'd do differently

Interview Flow

Start with a warm greeting. Acknowledge that they built a working platform that real users are on — that's a genuine achievement. Express curiosity about how they approached it and what they learned along the way.

Work through the questions naturally. DO NOT read them as a list — weave them into conversation, follow interesting threads, and ask follow-ups when something is worth exploring deeper. If they get passionate about a particular technical decision or challenge, stay there.

The interview has a natural arc: start personal (who are you, how did you approach this), move through the system (architecture, data, frontend), then look ahead (constraints, risks, what would need to change).

Part 1: The Builder and the Build (from Journalist + Architect)

- Tell me about yourself — what's your background? How did you come to be building The Grafter platform?
- When the project started, what was the brief? What were you asked to build, and how much freedom did you have in the technical decisions?
- Walk me through the early days — how did you go from Rachel's methodology (the playbook, the slides) to a working platform? What was the first thing you built?
- What's the tech stack? Take me through the layers — frontend framework, backend, hosting, any third-party services or APIs.
- How would you describe the architecture in a sentence or two? Is it a monolith, microservices, serverless, something else?

- Is there an architecture diagram? Even a rough one on a napkin? If you were drawing the system on a whiteboard right now, what would it look like?

Part 2: The Data Layer (from Architect)

- Let's talk about the database. What are you using — Firebase, PostgreSQL, MongoDB, something else? What drove that choice?
- How is the data model structured? Walk me through the main entities — users, companies, assessments, recommendations — and how they relate to each other.
- If I'm a user and I complete an assessment, what actually gets stored? What does that data look like under the hood?
- How are recommendations generated? Is the logic rule-based, AI-driven, a mix? Where does that logic live in the codebase?
- The team has heard that data export is currently HTML-only. Is that accurate? What would it take to expose the underlying data as JSON?
- Is there an API layer? Can anything access the data programmatically today, or is everything through the UI?
- How are backups handled? What's the recovery story if something goes wrong?

Part 3: The AI Integration (from Architect)

- The platform uses generative AI to help produce advice and outputs. Walk me through how that works — which models, which providers, how are prompts structured?
- Where does the AI sit in the flow? Does it process user inputs in real time, or is it batch/offline?
- How do you handle prompt engineering? Are prompts hardcoded, templated, dynamic? Who maintains them?
- Have you run into any issues with AI quality, hallucination, or consistency? How do you handle that?
- If the platform were to shift toward extracting data from URLs, LinkedIn profiles, or voice memos — rather than asking users to fill forms — how much of the current AI setup could support that?

Part 4: Authentication, Users, and the Community (from Architect)

- How do users authenticate? Email/password, OAuth, SSO, something else?
- What user roles and permissions exist? Is there an admin layer?
- How many registered users does the platform have right now? Roughly how many are active?
- The community directory — how are profiles structured? What data is collected during onboarding, and where does it live?
- Is there any matching or connection logic between community members today, or is the directory purely a listing?
- If someone wanted to auto-populate a profile from a LinkedIn URL or other public source, where would that fit in the current architecture?

Part 5: Frontend and UX (from Architect + Journalist)

- What's the frontend built with? React, Vue, vanilla JS, something else?
- How is the assessment/diagnostic flow built? Is it a single long form, multi-step wizard, something dynamic?
- How is state managed on the client side?
- Is the platform responsive / mobile-friendly?
- What are the UX areas you're least happy with? Where do users struggle?
- If you could rebuild one part of the frontend from scratch tomorrow, what would it be and why?

Part 6: Development and Operations (from Architect)

- What does your development workflow look like day-to-day? Branching strategy, PR process, testing?
- Is there a staging or test environment, or does everything go straight to production?
- What testing exists — unit, integration, end-to-end? What's the coverage like?
- How are secrets and configuration managed?
- How often do you deploy? What does a release look like?
- Is there any CI/CD pipeline? What does it look like?

Part 7: Constraints, Risks, and the Road Ahead (from Architect + Journalist)

- What are the biggest technical risks or pain points in the platform today? The things that keep you up at night, or that you know are fragile.
- Are there any licensing constraints on the codebase or dependencies that would affect open-sourcing?
- If someone said "we want to make the entire codebase open source" — what would need to happen? What's the gap between where you are now and open-source-ready?
- Is there documentation beyond the code? Architecture Decision Records, wikis, READMEs, inline comments?
- What are you most proud of in what you've built? What's the part that works really well that maybe doesn't get enough credit?
- If you had unlimited time and resources, what would you build next for the platform?
- Is there anything about the system that you think people should understand but probably don't? Any hidden complexity or context that's important?

Output Format

After the interview, produce a clean markdown file with the following structure:

---
title: "Technical Discovery: The Grafter Platform — Current Architecture"
date: 2026-02-17
type: interview
topics: [architecture, data-model, tech-stack, ai-integration, operations, open-source-readiness]
subject: Current Technical Lead
interviewer: LLM Voice Mode (on behalf of Architect and Journalist roles)

Technical Discovery: The Grafter Platform

Date: 2026-02-17
Subject: Current Technical Lead
Interviewer: LLM Voice Mode (on behalf of the Architect and Journalist)

Key Takeaways
[5-7 bullet points summarising the most important architectural findings — quotes where possible]

Architecture Summary
[A concise written description of the system architecture as described in the interview. Include: stack, hosting, data layer, AI integration, authentication. This should read as a standalone summary someone could skim in 2 minutes.]

Part 1: The Builder and the Build
[How the platform came to be, technical background, early decisions. Direct quotes.]

Part 2: The Data Layer
[Database, schema, entities, relationships, export capabilities. Direct quotes.]

Part 3: The AI Integration
[Models, providers, prompt architecture, quality handling. Direct quotes.]

Part 4: Authentication, Users, and the Community
[Auth, roles, user numbers, directory structure. Direct quotes.]

Part 5: Frontend and UX
[Framework, assessment flow, state management, known pain points. Direct quotes.]

Part 6: Development and Operations
[Workflow, testing, CI/CD, deployment, environments. Direct quotes.]

Part 7: Constraints, Risks, and the Road Ahead
[Technical debt, licensing, open-source readiness, pride points, aspirations. Direct quotes.]

Architecture Diagram (Verbal)
[Based on the interview, a text-based representation of the architecture as described. Use ASCII art, mermaid notation, or structured text. Note: this is interpreted from conversation — should be validated with the Technical Lead.]

Open-Source Readiness Assessment
[Based on what was described, a preliminary assessment of what would need to change to make the platform open-source-ready. Categories: Blockers, Significant Effort, Straightforward, Already There.]

Action Items for the Team
[Concrete follow-ups: documents to request, areas needing deeper investigation, decisions to be made.]

Signals and Insights
[Patterns, surprises, or strategic observations from the conversation — tone shifts, areas of pride, areas of concern, moments of strong conviction or hesitation.]

Keep the transcript conversational but structured. Use direct quotes generously. Note when the subject seemed particularly emphatic, proud, or concerned about something. Flag any information that changes assumptions from the strategic briefing.

Post-Interview Workflow

1. The Technical Lead copies the markdown output from the LLM
2. The Technical Lead shares it with Rachel and the project team
3. The team processes the interview into:
    - **Architect**: architecture assessment, gap analysis against proposed evolution, dependency map

- **Journalist**: narrative for the project history ("Where We Started")
- **Cartographer**: input for Wardley Map positioning of current platform components
- **Librarian**: technical knowledge base — stack, patterns, constraints
- **Historian**: decision record for architectural context and rationale

4. The Technical Lead receives a follow-up with any clarification questions
5. If they mentioned or drew an architecture diagram, that gets digitised and added to the repo

---

Why This Interview Matters

This interview is the foundation for every technical recommendation that follows. Without understanding the current system — not just what it does, but why it was built that way — any proposals for evolution risk being disconnected from reality.

The Technical Lead built a working platform that real users are on. That context — the constraints they worked within, the trade-offs they made, the things they'd do differently — is as valuable as any architecture diagram. Capturing it now, while it's fresh and while the Technical Lead is engaged, prevents the team from making assumptions that could be expensive to correct later.

The output feeds directly into:

- **Architecture proposals** for the platform evolution
- **Open-source readiness assessment** for the GitHub publication strategy
- **Migration planning** if the data layer or frontend needs to change
- **Risk register** for technical debt and dependencies

Prepared by Architect & Journalist Roles | The Cyber Boardroom | v0.1.0